

Current best Julia v1.5 SMC²-FC config (RTX 5090) — v2

2026-05-09 update: filter HMC unified with framework ChEES, fast controller is the new default

Generated 2026-05-09

What changed since v1

The 2026-05-08 v1 of this guide documented an out-of-date config. The 2026-05-09 changes that prompted this v2:

- **Filter HMC was 0 % accept across 2880 logged tempering levels** in the v1 saturated baseline (measured: `source compare_v15_julia_vs_python/example_run/julia_horizon_sweep_2026-05-09/` bench logs; cross-tabulation in `TIER_0_FINDINGS.md`). Two underlying bugs: step size ε too large and h_{fd} too small (Bug A); MCMC kernel did not scale by λ (Bug B, masked by Bug A). *Fixed* by porting the filter HMC to the framework's generic ChEES path (`parallel_hmc_one_move_generic!` + `chees_pick_L_generic` in `julia/SMC2FC_functional/src/Control/GPU` passing `beta = next_lambda`, with $\varepsilon = 0.005$ and $h_{fd} = 0.01$).
- **Fast controller is the new default** (`--ctrl-n-smc 1024 --ctrl-num-mcmc 8`; was `2048 / 16` in v1). The technical guide v1's "When to deviate $\rightarrow$ shorter wall time" recipe is now the default; saturated controller is the deviation.
- **New filter-side rejuvenation knobs** that v1 did not expose at all: `--liu-west-a 0.97, --smooth-resample-bw 1.0, --gaussian-bridge true, --h-fd 0.01, --filter-chees-min/max 4 / 64, --collect-ctrl-diagnostics true`.

Compared to the v1 saturated config, the new defaults at $T = 28$ d trade roughly the same wall (19.1 vs 19.5 min) for a filter HMC that actually works (mean accept 65.5 % vs 0 %). Full table in §7.

TL;DR — minimal CLI invocation

The new defaults reproduce the saturated 2026-05-09 unified-filter-HMC sweep with no flag overrides:

```
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
```

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \  
    --T-days 28 --seed 42 --output-dir /tmp/v15_T28d_run
```

Verified on a sanity run 2026-05-09 (saved manifest at `/tmp/v15_new_defaults_T2d/manifest.json`): a bare invocation produces `hmc_step_size = 0.005, h_fd = 0.01, liu_west_a = 0.97, smooth_resample_bw = 1.0, gaussian_bridge = true, filter_chees_min/max = 4 / 64, controller n_smc = 1024 num_mcmc = 8`.

If you need to be explicit about every flag (for reproducibility / scripts), the full command is:

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \  
    --T-days 28 --seed 42 \  
    --num-mcmc 3 \  
    --ctrl-n-smc 1024 --ctrl-num-mcmc 8
```

```

--hmc-step-size 0.005 \
--hmc-leapfrog 4 \
--h-fd 0.01 \
--filter-chees-min 4 --filter-chees-max 64 \
--liu-west-a 0.97 \
--smooth-resample-bw 1.0 \
--gaussian-bridge true \
--N-smc 512 --K-per-chain 1000 \
--ctrl-n-smc 1024 --ctrl-num-mcmc 8 \
--collect-ctrl-diagnostics true \
--output-dir /tmp/v15_T28d_run

```

The two are equivalent at the time of writing (verified by reading the bench’s `_parse_args` defaults dict in `version_1.5_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl`).

Contents

1	The framework: SMC2FC_functional	2
2	Filter-side config (the inner PF + outer SMC²)	2
3	Filter-side rejuvenation knobs	3
4	Filter HMC = framework ChEES path	4
5	Controller-side config (the SMC²-MPC plan finder)	4
6	Why this config (not bigger, not smaller)	5
7	Headline scientific result at the new defaults	5
8	When to deviate from these defaults	6
9	Where artefacts live	6

1 The framework: SMC2FC_functional

`julia/SMC2FC_functional/` is the default Julia framework. The previous `julia/SMC2FC/` has been renamed to `julia/SMC2FC_DEPRECATED_USE_FUNCTIONAL/` and is no longer used by the v1.5 bench. Both the bench’s `Project.toml` and `version_1.5_Julia/models/fsa_high_res/gpu_pf.jl` import `SMC2FC_functional` (commits 4a5c007, 9bab259; numerics bit-identical to the deprecated framework on the GPU bench). v2 of this guide adds: the bench’s filter HMC also goes through the framework’s generic ChEES code path (§4).

2 Filter-side config (the inner PF + outer SMC²)

Why $\varepsilon = 0.005$? The standalone HMC accept-rate sweep at `compare_v15_julia_vs_python/example_run/julia_horizon_sweep_2026-05-09/diag_hmc_accept_sweep.csv` varied $\varepsilon \in [10^{-3}, 0.2]$, $L \in \{1, 4, 16\}$, $h_{fd} \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ at fixed $M = 128$ chains over a 1-day truth-window prior cloud. Selected rows:

Pick: $\varepsilon = 0.005$, $h_{fd} = 10^{-2}$. The rolling-window unified-HMC sweep (`julia_horizon_sweep_2026-05-09/filter_hmc_accept_sweep.csv`) measured 61.9–71.4% mean accept at this default across $T \in \{14, 28, 42\}$ d (`docs/filter_hmc_accept_summary.csv`).

flag	value	what it controls
--N-smc	512	outer SMC ² posterior particles over θ (10-D filter parameters)
--K-per-chain	1000	inner-PF state particles per chain (latent state cloud size)
--num-mcmc	3	filter HMC moves per outer tempering level
--hmc-step-size	0.005	filter HMC ε (was 0.05 in v1; sweet-spot from sweep)
--hmc-leapfrog	4	filter HMC initial-fallback L (overridden adaptively by ChEES)
--h-fd	0.01	filter HMC FD-gradient step (was hardcoded 10^{-3} ; new in v2)
--filter-chees-min	4	filter ChEES candidate-list minimum (new in v2)
--filter-chees-max	64	filter ChEES candidate-list maximum (list = [4, 8, 16, 32, 64]; new in v2)
--max-temp-levels	30	filter tempering cap (typically uses 5 in practice)
--max-lambda-inc	0.20	maximum $\Delta\lambda$ per step
--target-ess-frac	0.5	ESS-bisection target as fraction of N_{smc}
--ot-max-weight	0.0	OT rescue OFF; turning it on costs $\sim 12\times$ wall

Table 1: Filter-side defaults. Bold-equivalent changes from v1: `--hmc-step-size` $0.05 \rightarrow 0.005$, and the new flags `--h-fd`, `--filter-chees-min`, `--filter-chees-max`.

ε	L	h_{fd}	accept (%)
0.001	1	10^{-2}	73.4
0.001	4	10^{-2}	74.2
0.005	1	10^{-2}	58.6
0.005	4	10^{-2}	36.7
0.01	1	10^{-2}	50.0
0.05	4	10^{-3} (v1 defaults)	0.0

Table 2: Selected rows from `diag_hmc_accept_sweep.csv`.

3 Filter-side rejuvenation knobs

These flags wire in three complementary diversification mechanisms on the parameter cloud. v1 had none of them exposed; v2 makes them the default.

flag	value	where it fires
--liu-west-a	0.97	per-tempering level, between systematic resample and HMC; per-dim Liu–West shrinkage
--smooth-resample-bw	1.0	per-tempering level; Silverman-bandwidth KDE resample with Liu–West correction
--gaussian-bridge	true	between rolling windows; multivariate Gaussian bridge

Table 3: New filter-side rejuvenation defaults. All three are ON by default in v2.

Per-dim Liu–West shrinkage (`--liu-west-a` 0.97): $\theta_i \leftarrow a \cdot \theta_i + (1 - a) \cdot \bar{\theta} + \sqrt{1 - a^2} \cdot \sigma_d \cdot \xi$ applied per parameter dimension, between systematic resample and HMC. Inline implementation in `version_1.5_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl` inside `run_outer_smc`. Fall-back; superseded by smooth-resample when that is on.

Silverman+KDE+LW smooth resample (`--smooth-resample-bw` 1.0): multivariate kernel-density resample with bandwidth from Silverman’s rule and a coupled Liu–West correction $a = \sqrt{1 - h_{\text{norm}}^2}$. Math ported into the bench from `julia/SMC2FC_functional/src/Filtering/Kernels.jl::smooth_resample`. When `smooth_resample_bw > 0` this branch is selected over the per-dim Liu–West.

Multivariate Gaussian bridge (`--gaussian-bridge` true): between rolling windows, replaces the identity-copy of the previous-window posterior with a sample from $\mathcal{N}(\hat{\mu}, \hat{\Sigma}_{\text{reg}})$ via `Bridge.fit_gaussian` + `Bridge.sample_from_gaussian` in `julia/SMC2FC_functional/src/SMC2/Bridge.jl`. Regularised sample covariance with 10^{-6} Tikhonov.

4 Filter HMC = framework ChEES path

In v1 the filter HMC was a model-specific function (`parallel_hmc_one_move!`) in `models/fsa_high_res/gpu_pf.jl` with `FIXED` leapfrog count and a `tempered_grads` closure that did not depend on λ . In v2 the filter calls the same generic framework functions the controller uses:

- `parallel_hmc_one_move_generic!` in `julia/SMC2FC_functional/src/Control/GPUControlSMC.jl`. Takes a `beta::Float64` keyword and computes `vals_prior + beta * vals_data` (and the corresponding gradient) — this fixes Bug B (missing λ -tempering) by construction. Bench passes `beta = next_λ` per tempering level.
- `chees_pick_L_generic` same file. Adapts the leapfrog count L per tempering level by maximising expected squared jump distance per $\varepsilon \cdot L$ across the candidate list $\{4, 8, 16, 32, 64\}$ (filter side; controller list is $\{16, \dots, 512\}$).

The bench’s call site in `run_outer_smc` now wraps `gpu_log_density` as a closure `log_density_fn :: Matrix{Float64} -> Vector{Float64}` and passes it to the framework functions. The model-specific `parallel_hmc_one_move!` and `parallel_hmc_one_move` in `gpu_pf.jl` are no longer called by the bench (left in place, separate cleanup).

Measured filter HMC behaviour at the new defaults. From the unified-HMC sweep (`julia_horizon_sweep.jl`, `docs/filter_hmc_accept_summary.csv`):

T (d)	n levels	mean accept (%)	median (%)	min (%)	max (%)	mean ChEES L
14	130	71.35	73.00	24	94	58.22
28	270	65.47	68.00	13	94	54.70
42	410	61.90	64.00	10	93	51.84

Table 4: Measured filter HMC behaviour. Compare with the v1 configuration’s **0 % accept across 2880 levels**.

5 Controller-side config (the SMC²-MPC plan finder)

flag	value	what it controls
<code>--ctrl-n-smc</code>	1024	outer SMC ² particles over RBF θ_{ctrl} (was 2048 in v1)
<code>--ctrl-num-mcmc</code>	8	ChEES-HMC moves per controller tempering level (was 16 in v1)
<code>--ctrl-chees-max</code>	512	ChEES picker upper bound; candidate list = $[16, 32, 64, \dots, 512]$
<code>--ctrl-n-anchors</code>	12	RBF basis cardinality (controller posterior dimension)
<code>--ctrl-max-levels</code>	40	cap on controller tempering levels (typically uses 5–6)
<code>--ctrl-n-inner</code>	64	inner CRN-MC trajectories per cost evaluation
<code>--ctrl-target-nats</code>	8.0	auto-calibration target entropy (nats)
<code>--ctrl-sigma-prior</code>	1.5	prior std on the RBF coefficient vector
<code>--ctrl-hmc-step</code>	0.2	base leapfrog step size ε
<code>--ctrl-hmc-leap</code>	16	base leapfrog count (overridden adaptively by ChEES)
<code>--collect-ctrl-diagnostics</code>	true	per-tempering-level controller HMC diagnostics CSV (new in v2)

Table 5: Controller-side defaults. Changes from v1: `--ctrl-n-smc` 2048 $\rightarrow$ 1024, `--ctrl-num-mcmc` 16 $\rightarrow$ 8, plus the new `--collect-ctrl-diagnostics` flag (default `true`). To recover v1’s saturated config, pass `--ctrl-n-smc` 2048 `--ctrl-num-mcmc` 16.

Controller-HMC measured behaviour at the new defaults (`docs/controller_hmc_summary.csv`):

T (d)	replans	total levels	mean accept (%)	mean L	mean $\beta_{\max}$	mean ESJD/ εL
14	13	69	96.87	16	226.39	14.12
28	27	147	95.43	16	27.02	13.73
42	41	238	96.31	16	5.69	13.63

Table 6: Controller HMC was already correct in v1 (the β keyword in `parallel_hmc_one_move_generic!` has always been respected on the controller side). v2’s instrumentation just makes this measurable per stride.

6 Why this config (not bigger, not smaller)

The filter-HMC tier is now correct, not just larger. v1 reached its filter “saturated” tier by setting $N_{\text{smc}} = 512$ and $K_{\text{per_chain}} = 1000$ but left the HMC kernel rejecting 100 % of proposals — so the cloud was kept alive only by systematic resample. v2 re-enables HMC by porting it to the framework’s working code path; the filter cloud now mixes each tempering level (61.9–71.4 % accept, Table 4).

The fast controller is the new default because it delivers comparable wall to v1’s saturated config ($T = 28$ d: 19.1 vs 19.5 min) while the filter side does the extra HMC work. Saturated controller is now an opt-in deviation (§8).

Smaller still leaves the GPU empty. The historical $N = 32, K = 200$ default mentioned in v1 stays a footnote: GPU util ~ 38 %, controller settles for a weakly-ramped schedule.

Bigger is still wall-time-infeasible for now. v1 noted $N = 1024, K = 4000, n_{\text{anchors}} = 16$ does not produce log output in 33 min wall. Untouched in v2.

7 Headline scientific result at the new defaults

metric ($T = 28$ d, seed=42, RTX 5090)	v1 saturated config	2026-05-09 default (v2)
total wall time	19.5 min	19.1 min
mean GPU util	49.5 %	59.0 %
peak VRAM	6662 MiB	5903 MiB
filter HMC mean accept	0 %	65.5 %
controller HMC mean accept	(not measured)	95.4 %
applied $\bar{\Phi}$ at stride 21	0.695	1.72
$\bar{A}_{\text{MPC}}$	(not in v1)	0.090
final A_{MPC}	(not in v1)	0.155
F violations	(not in v1)	0

Table 7: Comparison of v1’s saturated baseline with the new v2 default. Sources: v1’s quoted numbers from `compare_v15_julia_vs_python/example_run/`
`julia_saturation/FINDINGS_T28d_julia_saturated.md`; v2 numbers from
`julia_horizon_sweep_2026-05-09_filter_hmc_unified/`
`horizons_results.csv` + `docs/filter_hmc_accept_summary.csv` +
`docs/controller_hmc_summary.csv`.

What the table is saying. The new defaults’ wall is within 2 % of v1’s saturated config wall, but the filter HMC contributes real mixing (65.5 % accept) where v1’s was producing zero useful HMC work. The applied $\bar{\Phi}$ at stride 21 jumps from 0.695 to 1.72 — the controller is acting on a more informative posterior, so it commits more aggressively to the build-up phase. Closed-loop $\bar{A}_{\text{MPC}}$ at $T = 28$ d is 0.090 (vs the constant- $\Phi = 1$ baseline rollout’s 0.181); zero F -barrier violations.

8 When to deviate from these defaults

- **Want the v1 saturated controller back** (richer Φ schedule basis, $\sim 4\times$ more wall): set `--ctrl-n-smc 2048 --ctrl-num-mcmc 16`. v1's "Why this config" discussion of the saturated tier still applies, just at the cost of $\sim 4\times$ wall.
- **Want the v1 filter HMC behaviour back** (debugging purposes only — this is the broken config): set `--num-mcmc 0` to disable filter HMC, OR `--hmc-step-size 0.05` to put it back in the 0% accept regime. Both reproduce the filter posterior collapse measured in `TIER_0_FINDINGS.md`.
- **Want all rejuvenation tricks off** (also debugging): set `--liu-west-a 0.0 --smooth-resample-bw 0.0 --gaussian-bridge false`. The cloud will collapse by day ~ 15 per the no-filter-HMC baseline measurements.
- **Need OT rescue** (very low ESS / hard observation models): set `--ot-max-weight 0.01` and accept the $\sim 12\times$ slowdown. v1.5's 3-channel direct-Gaussian observation model has not needed this so far.
- **Want even shorter wall for debug runs**: there is no "safer" controller tier below 1024 / 8 in current testing. Drop `--N-smc` to 64 and `--K-per-chain` to 200 if you need a ~ 30 s sanity test — but the filter posterior is no longer scientifically meaningful at that scale.
- **Reproducibility**: always set `--seed` explicitly. Default is 42. Keep `--replan-K 2` unless you specifically want more or less frequent re-planning.

9 Where artefacts live

- **Bench script**: `version_1_5_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl`.
- **Sweep launchers**: `version_1_5_Julia/tools/launchers/run_julia_horizon_sweep_filter_hmc_unified.sh` (the new default config sweep) and the original `run_julia_horizon_sweep.sh`.
- **Plotting and diagnostics**: `compare_v15_julia_vs_python/src/`. This now includes `diag_cloud_collapse.jl`, `diag_controller_hmc.jl`, `diag_filter_hmc_accept.jl`, `diag_posterior_median_compare.jl`.
- **Microbench**: `version_1_5_Julia/tools/profile_gpu.jl --both` for filter + controller per-call timings. Unchanged from v1.
- **GPU PF tests**: `version_1_5_Julia/tools/test_gpu_pf.jl`. Unchanged from v1.
- **Lean4 diff test**: `version_1_5_Julia/diff_test/test_lean_diff_v15.jl`. Unchanged from v1.
- **2026-05-09 measurement runs**:
 - v1 saturated baseline: `compare_v15_julia_vs_python/example_run/julia_horizon_sweep_2026-05-09/` (with `TIER_0_FINDINGS.md`).
 - No-filter-HMC sweep (rejuvenation-only baseline): `compare_v15_julia_vs_python/example_run/julia_horizon_sweep_2026-05-09_no_filter_hmc/` (with `HMC_NOT_NEEDED_FINDINGS.md`).
 - Unified filter-HMC sweep (the new default config): `compare_v15_julia_vs_python/example_run/julia_horizon_sweep_2026-05-09_filter_hmc_unified/` (with `docs/filter_hmc_unified_sweep_written`).
- **Framework**: `julia/SMC2FC_functional/` (the default; *do* import from this). Depreciated framework at `julia/SMC2FC_DEPRECATED_USE_FUNCTIONAL/` (do *not* import from this).